STM32 EMBEDDED FIRMWARE | REGISTER-LEVEL DRIVERS

Reusable Bare-Metal I2C Functions

A practical function reference for direct-register STM32 projects: bus setup, transactions, register access, buffers, timeouts and error handling.

I2C Protocol	Bare metal No HAL calls	Reusable API Driver + application

What you will learn

Use this guide as a reusable function reference when building embedded projects in STM32CubeIDE. Keep the low-level bus driver separate from application-specific work such as reading a sensor, controlling a motor or storing data.

Common functions

I2C_Init() - Configure pins, peripheral clock, bus speed and peripheral enable.

I2C_WaitFlag() - Wait for a hardware event with a timeout.

I2C_Start() - Generate a START condition and wait for master-mode status.

I2C_SendAddress() - Send a 7-bit target address with read or write direction.

I2C_WriteByte() - Send one data byte and check for ACK/error.

I2C_ReadByteAck() / I2C_ReadByteNack() - Read data and acknowledge intermediate/final bytes appropriately.

I2C_Stop() - Generate STOP and release the bus.

I2C_WriteRegister() / I2C_ReadRegister() - Common register-based sensor/IC operations.

I2C_WriteBuffer() / I2C_ReadBuffer() - Transfer multiple bytes.

I2C_GetError() / I2C_BusRecover() - Report faults and recover where hardware/design allow.

Important: I2C controller registers and event sequences vary by STM32 family. The API concepts are reusable, but implement the register operations from the reference manual for your exact MCU. STM32F1 uses an older I2C peripheral sequence; do not copy timing/flag code blindly to newer families.

1. Understand I2C

I2C is a synchronous, addressed, two-wire bus. SDA carries data and SCL carries the clock. A master normally initiates transfers; devices acknowledge bytes with ACK or NACK. SDA and SCL are open-drain signals and normally require pull-up resistors to a suitable logic supply.

START	Address + W/R	Register/data bytes	STOP
Begin transaction	Select target and direction	Transfer payload; check ACK	Release bus

2. Function list - what to reuse

Function	Responsibility	Reuse guidance
I2C_Init()	Enable clocks, configure GPIO open-drain alternate-function, timing and enable peripheral.	Reuse API; register setup is MCU-specific.
I2C_WaitFlag(flag, state, timeout)	Poll a status/event flag without hanging forever.	Reusable pattern; flags differ by family.
I2C_Start()	Generate START and verify master-mode/event status.	Core transaction helper.
I2C_SendAddress(addr7, direction)	Transmit target address and select read/write.	Keep addr7 in API; shift only where the peripheral format requires it.
I2C_WriteByte(byte)	Write one byte and detect ACK/NACK or bus error.	Core data helper.
I2C_ReadByteAck()	Read a byte and ACK when more bytes will follow.	Core receive helper; exact sequence is MCU-specific.
I2C_ReadByteNack()	Read final byte and NACK to finish a read.	Core receive helper.
I2C_Stop()	Issue STOP and wait for the bus/peripheral to settle.	Core transaction helper.
I2C_WriteRegister() / I2C_ReadRegister()	Write/read a device register using address and register index.	Reusable device-access layer.
I2C_WriteBuffer() / I2C_ReadBuffer()	Transfer a byte array of specified length.	Reusable for EEPROMs, sensors and RTCs.
I2C_GetError() / I2C_BusRecover()	Expose NACK/timeout/bus errors; optional GPIO clock recovery.	System-dependent; recovery must be designed safely.

3. Typical register-read transaction

A common example is reading one register from a sensor. The exact events and ACK handling must follow the STM32 family reference manual. This sequence describes the bus-level operation.

Step	Bus action	Purpose
1	START	Take control of the bus.
2	Address + Write, ACK	Select the target to specify which register to read.
3	Register address, ACK	Set the target device register pointer.
4	Repeated START	Change direction without releasing the bus.
5	Address + Read, ACK	Request data from the target.
6	Read data, NACK final byte	Signal that no more bytes are requested.
7	STOP	End transaction.

Example reusable API (interface only)

```
typedef enum { I2C_OK, I2C_NACK, I2C_TIMEOUT, I2C_BUS_ERROR } I2C_Status;

void I2C_Init(void);
I2C_Status I2C_WriteRegister(uint8_t addr7, uint8_t reg, uint8_t value);
I2C_Status I2C_ReadRegister(uint8_t addr7, uint8_t reg, uint8_t *value);
I2C_Status I2C_WriteBuffer(uint8_t addr7, const uint8_t *data, size_t len);
I2C_Status I2C_ReadBuffer(uint8_t addr7, uint8_t *data, size_t len);
```

Why return a status code?

A reusable driver should tell the caller whether the operation succeeded. A sensor application can retry after a timeout, report a missing device after NACK, or enter a safe state after a bus error. Avoid infinite waits: every flag wait should use a timeout.

Address convention

Use a consistent public API convention: accept the unshifted 7-bit target address (0x00-0x7F). Convert it to the register/peripheral-specific format inside the driver. Datasheets and libraries sometimes display an 8-bit address byte including the R/W bit, so check the convention.

4. Portable design vs MCU-specific code

Usually reusable	Must be checked for each STM32
Public function names and return codes; buffer APIs; timeout policy; error reporting; device register abstraction.	RCC clock enables; I2C instance and pins; alternate-function settings; timing/CCR/TRISE or TIMINGR registers; START/STOP/ACK event sequence; flag clearing; errata.

5. Common I2C problems to troubleshoot

Symptom	Things to check
No ACK	Wrong address, device unpowered, missing common ground, wiring error, incorrect address-pin levels.
Bus stuck busy	SDA or SCL held low, peripheral state fault, device mid-transaction; investigate before attempting recovery.
Wrong data	Wrong register pointer, wrong address convention, incorrect repeated-START or byte ACK/NACK handling.
Intermittent faults	Missing/incorrect pull-ups, excess bus capacitance, long wires, wrong bus speed or poor signal integrity.
Firmware hangs	Flag waits without timeouts, error flags not cleared correctly, or family-specific event ordering ignored.

6. Recommended project structure

```
Core/Inc/i2c_driver.h     // API, enums and documented address convention
Core/Src/i2c_driver.c     // registers, timing, transfer, timeout, error handling
App/Src/sensor.c          // sensor-specific registers and conversions
Core/Src/main.c           // initialize the driver and run application tasks
```

Official references

STMicroelectronics RM0008 - STM32F1 reference manual (I2C chapter):
https://www.st.com/resource/en/reference_manual/cd00171190-stm32f101-103-105-107-stm32f100-series-armbased-32bit-mcus-stmicroelectronics.pdf

ST AN2824 - STM32F10xxx I2C optimized examples (polling, interrupt and DMA):
https://www.st.com/resource/en/application_note/an2824-stm32f10xxx-i2c-optimized-examples-stmicroelectronics.pdf

Use the reference manual and errata for your exact part. This guide is a reusable function design reference, not a drop-in I2C implementation for every STM32 family.